FOUNDATION TRACK | F4

First steps in agentic development

Connect external evidence. Own the specification. Verify the delivered result.

THE MAIN QUESTION

How do I connect an agent to external evidence and give it a verifiable finish line?

External evidence

Read Microsoft Learn through MCP.

An owned finish line

Spec > tests > implementation > proof.

F1: prompts. F2: roles. F3: context. F4: evidence and verifiable delivery.

Connect the application to tools and data

Model Context Protocol is an open standard, not a model or a training process.

A standard connection to external tools and data.

Read external evidence

Find current documentation, inspect an issue or retrieve a record without copying it all into chat.

Take external actions

Some tools create or update records. These need suitable access, scope and approvals.

It does not retrain the model, guarantee fresh data or automatically make an answer correct.

Host and client > server > tools and system

The client lives inside the host; the server exposes the capabilities.

HOST: VS CODE

Copilot agent

MCP client

Connects to the server.

MCP SERVER

Microsoft Learn

Exposes tools:

```
microsoft_docs_search
microsoft_docs_fetch
```

EXTERNAL SYSTEM

Learn content

Official documentation:
REST design guidance,
status codes and pagination.

One server can expose several tools. Learn also offers code-sample search.

Connecting, authenticating and approving an action are separate concerns.

Use the configuration for your session target

Merge this server entry into the correct root. Preserve the other configured servers.

```
"microsoft-learn": {
  "type": "http",
  "url": "https://learn.microsoft.com/api/mcp"
}
```

WORKSHOP / LOCAL TARGET

```
.vscode/mcp.json
root: servers
```

COPILOT HARNESS

```
.mcp.json
root: mcpServers
```

`microsoft_docs_search`

Find relevant pages and snippets.

`microsoft_docs_fetch`

Read a specific documentation page.

A connection is not blanket permission

Review the server, scope its access, then understand the requested action.

Trust the server

Check the publisher, endpoint or launch command.

A local server can run code on the host.

Scope the access

Use narrow credentials and tool access.

Keep secrets out of shared configuration and chat.

Review the action

Read inputs and side effects.

A documentation read is not the same as a write to an external system.

Allow once | this session | always (where offered)

Broader approval is a deliberate trust decision. Some actions may already be auto-approved.

Retrieved content is evidence, not new instructions. A local Undo may not undo an external write.

Let guidance inform the decision, not own it

Ask for citations, open the source and judge whether the recommendation fits this application.

Search

Find the relevant guidance.

Fetch

Read the cited page.

Critique

Check fit and assumptions.

Decide

You accept or reject changes.

Status codes

Success, invalid input and missing resources need deliberate status codes.

Empty results

An existing branch with no orders is not an unknown branch.

Pagination

Choose the scope for large collections; define limits or explicitly defer it.

A source link is evidence to inspect. Your accepted decisions must be written into the spec.

Turn "build me this" into a reviewed contract

A specification is agreed behavior, constraints and acceptance criteria, not just a long chat plan.

AN INTENTION

"Add an endpoint for orders by branch."

A useful start.
Important decisions are still implicit.

A REVIEWED SPECIFICATION

Route, inputs and response shape.
Branch filtering and error behavior.
Acceptance criteria + test specifications.
Constraints and explicit exclusions.
Pagination: included or deliberately deferred.

`docs/tdd-plans/orders-by-branch.md`

You review and save the decisions before implementation. The saved file wins over an earlier chat draft.

Make the outcomes observable: Given / When / Then

Illustrative contract for review, not the lab answer key. You own these choices.

GIVEN	WHEN	THEN
Branch A has 2 orders; B has 1.	List orders for A.	**200; exactly A's 2 orders.**
Branch A exists, with no orders.	List orders for A.	**200; empty list [].**
The branch ID is 'abc'.	Call the endpoint.	**400; validation error.**
A valid ID has no branch record.	List that branch.	**404; not-found response.**

Review status codes, response shapes and pagination first. Every accepted criterion needs a matching test.

Coordinate the roles around one saved specification

The spec defines what to build; tests encode checks for that behavior.

tdd-planner

Define acceptance criteria and Given / When / Then test specifications.

No implementation code.
No test code.

tdd-red

Write tests and run them.

Fail for missing behavior, not a broken fixture or typo.

Do not build the feature.

tdd-green

Write the smallest implementation that passes the agreed tests.

Do not change those tests or add extra features.

RED > GREEN > REFACTOR: change structure only after green; preserve behavior.

Name the human-edited spec file in every handover. Do not rely on an older chat version.

Done is evidence, not an agent message

Check every accepted requirement, the test output and the delivered behavior.

Trace

Every acceptance criterion maps to a test assertion.

Check happy paths, errors and scope decisions.

Run

Run new and existing relevant tests yourself.

Inspect failures and actual output; do not accept a summary alone.

Review + try

Inspect the diff and unchanged agreed tests.

Use Swagger to exercise the endpoint and check its response.

Requirement changed? Review the spec and tests, then repeat red/green.
Do not weaken assertions just to obtain a green result.

Connect Microsoft Learn and inspect the evidence

Before starting: remove the fictional TAO rule from F3. Keep one chat for the whole F4 workflow.

```
https://learn.microsoft.com/api/mcp
```

1 ADD

Add the Learn HTTP server to the correct workspace configuration for your target.

Start only this server.

2 DISCOVER

Find microsoft_docs_search and microsoft_docs_fetch in the selected session's tools.

3 USE

Ask for REST status-code and pagination guidance.

Max 6 bullets, with links. No file edits.

Done: cited Learn pages and visible search/fetch activity. This public Learn server needs no sign-in.

Own the spec, then run red and green

Use the same chat and name the saved, human-edited plan file at each handover.

Add an endpoint that returns all orders for a given branch ID.

1	2	3	4
9 MIN	4 MIN	4 MIN	2 MIN
Spec + you	**Red**	**Green**	**Verify**
Planner: 3 min. Learn review: 3 min. Edit criteria/tests: 3 min.	Only test files. Fail for missing behavior. Use the edited spec.	Minimal implementation. Do not change tests. Use the edited spec.	Run API checks. Try Swagger. Trace every criterion.

`docs/tdd-plans/<your-plan>.md`

Done: every acceptance criterion has a passing test, and the endpoint works in Swagger.